

Semestrální práce z předmětu KIV/UPS

Kevin Varchola

14. ledna 2025

Obsah

1	Popis hry	3
1.1	Pravidla hry	3
2	Popis protokolu	3
2.1	Formát zpráv	3
2.2	Přenášené typy požadavků/odpovědí	4
2.3	Příklady zpráv	5
3	Popis implementace	5
3.1	Použité knihovny a verze	5
3.2	Paralelizace	5
4	Struktura projektu	5
4.1	Klient	5
4.2	Server	6
4.3	Další soubory	6
5	Použití a spuštění programu	7
6	Závěr	7

1 Popis hry

”Hra na pistolky” je jednoduchá tahová hra pro dva hráče, kde každý hráč má možnost vybrat z několika akcí (např. shoot, reload, cover). Cílem hry je eliminovat soupeře.

1.1 Pravidla hry

Hra se hraje na tahy:

- Každý hráč má na začátku hry 3 životy a 1 náboj.
- Hráči simultánně vybírají své akce v každém tahu.
- Server po obdržení akcí od obou hráčů vyhodnotí jejich efekt a aktualizuje stav hry.
- Hráč může vybrat akce jako:
 - **Shoot** - vystřelí na soupeře (pokud má náboje).
 - **Reload** - přidá jeden náboj.
 - **Cover** - chrání se proti střelbě.
- Hra končí, když je jeden nebo oba hráči eliminováni.

2 Popis protokolu

Komunikace mezi klientem a serverem je realizována pomocí jednoduchého textového protokolu, který zajišťuje výměnu informací o stavu hry a akcích hráčů. Protokol je navržen tak, aby byl rozšiřitelný a umožňoval snadné přidávání nových typů zpráv.

2.1 Formát zpráv

Každá zpráva obsahuje typ požadavku nebo odpovědi, a to ve formátu klíč-hodnota odděleném znakem `&`:

- `request_type`: Typ požadavku odesílaného klientem na server.
- `response_type`: Typ odpovědi vrácené serverem klientovi.
- `status_code`: Kód odpovědi indikující úspěch nebo chybu.
- `message`: Doplňující zpráva nebo popis chyby.

2.2 Přenášené typy požadavků/odpovědí

Komunikace probíhá prostřednictvím několika typů zpráv, které jsou specifikovány níže:

- **ping:**
Slouží k otestování spojení mezi klientem a serverem. Klient odesílá zprávu **ping** a server na ni odpovídá zprávou **pong**.
- **game_ready:**
Oznámení, že hra je připravena k zahájení.
- **player_state:**
Obsahuje informace o aktuálním stavu hráče (např. počet životů a nábojů).
- **opponent_state:**
Informace o stavu soupeře (počet životů).
- **close_game:**
Zpráva od hráče, který chce ukončit hru.
- **reset_game:**
Požadavek na restartování hry, obvykle po jejím ukončení.
- **action:**
Akce provedená hráčem. Může se jednat o jednu z následujících akcí:
 - **shoot:** Hráč vystřelil.
 - **reload:** Hráč dobíjel.
 - **cover:** Hráč se kryl.
- **game_result:**
Výsledek hry, který zahrnuje informace o vítězi nebo informaci o remíze.
- **round_state:**
Stav aktuálního herního kola (např. zda je kolo aktivní nebo zda skončilo a čeká na nové akce).
- **game_state:**
Celkový stav hry (např. zda hra stále běží, zda byla přerušena, nebo je dokončena).
- **name:**
Nastavení jména hráče, které se používá při připojení ke hře.

2.3 Příklady zpráv

- `request_type=ping`
- `response_type=ping&status_code=200&message=Pong`
- `request_type=game_ready`
- `response_type=game_ready&status_code=200&message=Game ready`
- `request_type=action&action_type=shoot`
- `response_type=action&status_code=200&Action set`

3 Popis implementace

3.1 Použité knihovny a verze

- Klient: Python 3.11., knihovna `tkinter` pro GUI.
- Server: Go 1.23.

3.2 Paralelizace

Server využívá vlákna pro obsluhu jednotlivých klientů. Každý klient je zpracováván nezávisle, což umožňuje simultánní interakci s více hráči.

4 Struktura projektu

Projekt je rozdělen do dvou hlavních složek: `client/` a `server/`. Každá složka obsahuje specifické soubory pro danou část aplikace.

4.1 Klient

- `client.py`:
Hlavní logika klienta, která spravuje komunikaci se serverem.
- `connect_win.py`:
Modul pro zobrazení okna pro připojení k serveru.
- `const.py`:
Obsahuje konstanty a konfigurační parametry pro klienta.

- `game_win.py`:
Modul implementující hlavní herní okno a interakci uživatele.
- `main.py`:
Spouštěcí soubor klienta.
- `menu_win.py`:
Modul pro hlavní menu aplikace.

4.2 Server

- `main.go`:
Hlavní spouštěcí soubor serveru.
- `game.go`:
Implementace herní logiky na straně serveru.
- `const.go`:
Obsahuje konstanty a nastavení serveru.
- `server.go`:
Modul pro správu komunikace s klienty.
- `structures.go`:
Definuje datové struktury používané v aplikaci.
- `go.mod`:
Konfigurační soubor pro správu závislostí v jazyce Go.

4.3 Další soubory

- `Diagram.pdf`:
Stavový diagram protokolu.
- `Makefile`:
Automatizace spouštění klienta a serveru.
- `Protocol.txt`:
Dokumentace protokolu.

5 Použití a spuštění programu

Program se spouští pomocí `Makefile`, který zjednodušuje proces kompilace a spuštění.

1. Otevřete terminál a přejděte do složky projektu.
2. Ujistěte se, že máte nainstalované všechny potřebné závislosti.
3. Spuštění serveru:
 - Základní příkaz:
`make server` – spustí server na výchozí adrese `0.0.0.0:8080`.
 - S parametry:
`make server ip:port` – umožňuje specifikovat vlastní IP adresu a port (např. `make server 127.0.0.1:9090`).

Po úspěšném spuštění server vypíše zprávu potvrzující, že běží, a naslouchá na zadaném portu.

4. Spuštění klienta:
 - Příkaz: `make client`.
 - Po spuštění klient zobrazí hlavní menu aplikace, odkud se můžete připojit k serveru.

6 Závěr

Téma hry, kterou jsem si vybral, mě velmi bavilo. Během implementace jsem se naučil, jak funguje komunikace mezi serverem a klientem, a to jak v jazyce Go, tak v Pythonu. Projekt mi pomohl lépe pochopit paralelizaci a implementaci komunikačních protokolů. Při práci s paralelizací jsem se však setkal s řadou problémů, přičemž největší výzvou byly deadlocky, které bylo potřeba pečlivě ladit a vyřešit. Co se týče vizuální stránky, GUI by šlo určitě vylepšit, například přidáním animací a obrázků, které jsem původně plánoval implementovat, ale na které nezbyl čas. Celkově projekt přinesl cenné zkušenosti a otevřel nové oblasti ve vývoji.